



Proyecto Final Módulo 2

API REST para plataforma de alquiler de autocaravanas

Memoria técnica backend

CARLOS BELTRÁN ROMERO

Junio, 2026

Índice

1. Introducción	4
1.1. Objetivo del proyecto	4
1.2. Relación con la parte frontend	4
1.3. Tecnologías utilizadas	4
2. Análisis de requisitos	6
2.1. Requisitos generales	6
2.2. Recursos implementados	6
2.3. Usuarios y roles	6
3. Arquitectura del backend	7
3.1. Estructura general	7
3.2. Separación de responsabilidades	7
3.3. Flujo de una petición	8
4. Diseño de base de datos	9
4.1. Base de datos relacional: MySQL	9
4.2. Tabla users	9
4.3. Tabla vehicles	9
4.4. Base de datos no relacional: MongoDB	10
4.5. Diagrama de base de datos	10
5. Autenticación y autorización	12
5.1. Autenticación con Laravel Sanctum	12
5.2. Registro y login	12
5.3. Autorización por roles	12
5.4. Rutas públicas y privadas	13

6. Recursos de la API	14
6.1. Recurso Vehicles	14
6.1.1. Campos principales	14
6.1.2. Lógica de negocio	14
6.2. Recurso Reservations	15
6.2.1. Campos principales	15
6.2.2. Lógica de negocio	15
7. Validación de datos	16
7.1. Uso de Form Request	16
7.2. Validación de vehículos	16
7.3. Validación de reservas	17
8. Endpoints desarrollados	18
8.1. Autenticación	18
8.2. Vehicles	18
8.3. Reservations	18
9. Códigos HTTP utilizados	19
10.Instalación y ejecución del proyecto	20
10.1. Requisitos previos	20
10.2. Configuración de MySQL	20
10.3. Configuración de MongoDB	20
10.4. Comandos de instalación	21
11.Tutorial de pruebas con Thunder Client	22
11.1. Preparación	22
11.2. Prueba 1: listar vehículos	22
11.3. Prueba 2: detalle de vehículo	22
11.4. Prueba 3: consultar disponibilidad	22
11.5. Prueba 4: login como usuario normal	23
11.6. Prueba 5: consultar usuario autenticado	23

11.7. Prueba 6: crear reserva	23
11.8. Prueba 7: comprobar reserva solapada	24
11.9. Prueba 8: listar reservas del usuario	24
11.10 Prueba 9: actualizar reserva	24
11.11 Prueba 10: login como administrador	25
11.12 Prueba 11: crear vehículo como administrador	25
11.13 Prueba 12: comprobar autorización	26
11.14 Prueba 13: eliminar vehículo	26
12. Pruebas realizadas	27
12.1. Pruebas de rutas públicas	27
12.2. Pruebas de autenticación	27
12.3. Pruebas de autorización	27
12.4. Pruebas de reservas	27
12.5. Comprobación en bases de datos	27
13. Problemas encontrados y soluciones	28
13.1. Integración de MongoDB	28
13.2. Diferencia entre MySQL y MongoDB	28
13.3. Control de permisos	28
13.4. Solapamiento de reservas	28
14. Conclusiones y mejoras futuras	29
14.1. Conclusiones	29
14.2. Mejoras futuras	29

1. Introducción

1.1. Objetivo del proyecto

Este proyecto consiste en el desarrollo de una API REST para la plataforma VanGo, una aplicación orientada al alquiler de autocaravanas. La parte frontend ya había sido desarrollada previamente como una web con catálogo de vehículos, página de detalle y formulario de contacto o reserva. En este segundo módulo el objetivo ha sido construir la parte backend, de forma que la aplicación pueda trabajar con datos persistentes, autenticación de usuarios, autorización por roles y lógica real de reservas.

El propósito principal no ha sido adaptar directamente el frontend anterior al backend, sino desarrollar una API bien estructurada y suficientemente completa para que pudiera ser consumida por un cliente web en el futuro. Por ese motivo, el trabajo se ha centrado en diseñar correctamente los recursos, las rutas, la base de datos, la autenticación y las reglas de negocio.

1.2. Relación con la parte frontend

La temática del proyecto se mantiene respecto al módulo anterior. En la primera parte se simulaba una base de datos mediante un archivo JSON que contenía diferentes modelos de autocaravanas, como la Rapido 8094 DF, la Rapido 9094 DF, la Benimar Mileo 282 y la Bürstner Lyseo TD 744. En este backend esos datos pasan a tener una representación real en base de datos.

La API permite consultar vehículos disponibles, obtener el detalle de cada uno, comprobar disponibilidad en un rango de fechas y crear reservas. Además, se incorpora un sistema de usuarios con roles, de forma que un usuario normal puede crear y consultar sus reservas, mientras que un administrador puede gestionar la flota completa de vehículos.

1.3. Tecnologías utilizadas

Para el desarrollo del backend se ha utilizado Laravel como framework principal. La base de datos relacional se ha implementado con MySQL, gestionada visualmente mediante phpMyAdmin. Además, para cumplir con el uso de una base de datos no relacional, se ha integrado MongoDB para almacenar las reservas.

Las tecnologías principales han sido:

- **Laravel:** framework PHP utilizado para construir la API REST.
- **MySQL:** base de datos relacional usada para usuarios, vehículos y tokens de autenticación.

- **MongoDB:** base de datos documental usada para las reservas.
- **Laravel Sanctum:** sistema de autenticación mediante tokens.
- **Thunder Client:** herramienta utilizada para probar los endpoints.
- **MongoDB Compass:** interfaz gráfica para comprobar los documentos almacenados en MongoDB.
- **phpMyAdmin:** herramienta visual para comprobar las tablas MySQL.

2. Análisis de requisitos

2.1. Requisitos generales

El proyecto consiste en una API de temática libre. En este caso, la temática escogida es el alquiler de autocaravanas. La API debe cumplir los requisitos principales del módulo: uso de bases de datos relacionales y no relacionales, autenticación, rutas públicas y privadas, patrón REST, verbos HTTP adecuados, códigos de estado correctos, validación de datos, lógica de negocio y documentación de endpoints.

Al tratarse de un proyecto individual, se han implementado las funcionalidades obligatorias comunes y dos recursos principales relacionados con la temática:

- **Vehicles:** recurso para gestionar las autocaravanas disponibles.
- **Reservations:** recurso para gestionar las reservas realizadas por usuarios.

2.2. Recursos implementados

La API se ha diseñado alrededor de dos recursos principales. El primero es el recurso de vehículos, que representa la flota de autocaravanas de VanGo. El segundo es el recurso de reservas, que representa las solicitudes de alquiler realizadas por usuarios autenticados.

Recurso	Base de datos	Responsabilidad
Vehicles	MySQL	Gestionar la flota de autocaravanas
Reservations	MongoDB	Gestionar las reservas de usuarios
Users	MySQL	Autenticación y control de permisos

Cuadro 1: Recursos principales de la API

2.3. Usuarios y roles

La aplicación diferencia dos roles principales:

- **user:** usuario autenticado que puede crear, consultar, modificar y eliminar sus propias reservas.
- **admin:** usuario administrador que puede gestionar vehículos y consultar todas las reservas.

También existen rutas públicas, accesibles sin autenticación, como el listado de vehículos, el detalle de un vehículo, el registro y el login.

3. Arquitectura del backend

3.1. Estructura general

El proyecto sigue una estructura típica de Laravel para API REST. Las rutas se definen en `routes/api.php`, los controladores se encuentran dentro de `app/Http/Controllers/Api`, los modelos en `app/Models` y las validaciones se separan en clases `FormRequest` dentro de `app/Http/Requests`.

```
1 vango-api/  
2 |-- app/  
3 |   |-- Http/  
4 |   |   |-- Controllers/  
5 |   |   |   |-- Api/  
6 |   |   |   |   |-- AuthController.php  
7 |   |   |   |   |-- VehicleController.php  
8 |   |   |   |   |-- ReservationController.php  
9 |   |   |-- Middleware/  
10 |   |   |   |-- EnsureUserIsAdmin.php  
11 |   |   |-- Requests/  
12 |   |   |   |-- LoginRequest.php  
13 |   |   |   |-- RegisterRequest.php  
14 |   |   |   |-- StoreVehicleRequest.php  
15 |   |   |   |-- UpdateVehicleRequest.php  
16 |   |   |   |-- StoreReservationRequest.php  
17 |   |   |   |-- UpdateReservationRequest.php  
18 |   |-- Models/  
19 |   |   |-- User.php  
20 |   |   |-- Vehicle.php  
21 |   |   |-- Reservation.php  
22 |-- database/  
23 |   |-- migrations/  
24 |   |-- seeders/  
25 |   |-- factories/  
26 |-- routes/  
27 |   |-- api.php  
28 |-- config/  
29 |   |-- database.php  
30 |-- README.md  
31 |-- composer.json
```

3.2. Separación de responsabilidades

La API se ha organizado separando responsabilidades para evitar concentrar toda la lógica en un único punto. Las rutas solo se encargan de conectar endpoints con controladores.

Los controladores contienen la lógica de negocio principal. Los modelos representan las entidades y su conexión con la base de datos. Los `FormRequest` encapsulan las reglas de validación.

Esta separación permite que el código sea más fácil de leer, mantener y ampliar. Por ejemplo, las reglas de validación para crear vehículos no están mezcladas con la lógica de creación, sino que viven dentro de `StoreVehicleRequest`. De esta forma, el controlador recibe datos ya validados y puede centrarse en la lógica propia del negocio.

3.3. Flujo de una petición

El flujo general de una petición en la API es el siguiente:

```
1 Cliente HTTP
2   |
3   v
4 routes/api.php
5   |
6   v
7 Middleware de autenticación o admin
8   |
9   v
10 Form Request Validation
11  |
12  v
13 Controlador
14  |
15  v
16 Modelo Eloquent / Modelo MongoDB
17  |
18  v
19 Base de datos
20  |
21  v
22 Respuesta JSON con código HTTP
```

4. Diseño de base de datos

4.1. Base de datos relacional: MySQL

MySQL se ha utilizado para almacenar los datos más estructurados de la aplicación. En concreto, se almacenan los usuarios, los vehículos y los tokens de autenticación generados por Laravel Sanctum.

Las tablas principales en MySQL son:

- **users**: usuarios registrados en la aplicación.
- **vehicles**: autocaravanas disponibles en la plataforma.
- **personal_access_tokens**: tokens de autenticación de Sanctum.

4.2. Tabla users

La tabla **users** contiene los usuarios de la aplicación. Se ha añadido una columna **role** para diferenciar usuarios normales y administradores.

```
1 users
2 |-- id
3 |-- name
4 |-- email
5 |-- email_verified_at
6 |-- password
7 |-- role
8 |-- remember_token
9 |-- created_at
10 |-- updated_at
```

4.3. Tabla vehicles

La tabla **vehicles** representa la flota de autocaravanas. Se ha diseñado con campos suficientes para cubrir tanto el catálogo como la ficha de detalle.

```
1 vehicles
2 |-- id
3 |-- slug
4 |-- name
5 |-- brand
6 |-- price_per_day
7 |-- description
8 |-- images
```

```
9 |-- features
10 |-- capacity
11 |-- transmission
12 |-- fuel
13 |-- is_active
14 |-- created_at
15 |-- updated_at
```

Los campos `images` y `features` se almacenan como JSON en MySQL, ya que representan listas de valores asociadas a cada vehículo.

4.4. Base de datos no relacional: MongoDB

MongoDB se ha utilizado para almacenar las reservas. Esta decisión encaja bien con la naturaleza del recurso, ya que una reserva puede entenderse como un documento que agrupa información del usuario, del vehículo y del estado de la operación en un mismo registro.

La colección principal es:

- **reservations**: documentos de reservas.

```
1 reservations
2 |-- _id
3 |-- user_id
4 |-- vehicle_id
5 |-- vehicle_name
6 |-- start_date
7 |-- end_date
8 |-- days
9 |-- total_price
10 |-- status
11 |-- customer_name
12 |-- customer_email
13 |-- customer_phone
14 |-- notes
15 |-- created_at
16 |-- updated_at
```

4.5. Diagrama de base de datos

Aunque MySQL y MongoDB no comparten claves foráneas reales entre sí, la relación lógica de la aplicación es clara: un usuario puede crear muchas reservas y un vehículo puede aparecer en muchas reservas.

```
1  USERS
2      id PK
3      name
4      email
5      password
6      role
7
8  VEHICLES
9      id PK
10     slug
11     name
12     brand
13     price_per_day
14     capacity
15     transmission
16     fuel
17     is_active
18
19  RESERVATIONS_MONGO
20     _id PK
21     user_id
22     vehicle_id
23     vehicle_name
24     start_date
25     end_date
26     days
27     total_price
28     status
29
30  USERS 1 ---- N RESERVATIONS_MONGO
31  VEHICLES 1 ---- N RESERVATIONS_MONGO
```

5. Autenticación y autorización

5.1. Autenticación con Laravel Sanctum

La autenticación de la API se ha implementado mediante Laravel Sanctum. Este sistema permite generar tokens personales que se envían en las peticiones privadas mediante la cabecera `Authorization`.

Cuando un usuario hace login correctamente, la API devuelve un token. Ese token debe incluirse en las siguientes peticiones privadas.

```
1 Authorization: Bearer TOKEN_DEL_USUARIO
```

5.2. Registro y login

La API incluye endpoints para registrar usuarios y para iniciar sesión:

- POST /api/register
- POST /api/login

El registro crea siempre usuarios con rol `user`. El rol `admin` se crea mediante seeders, evitando que cualquier usuario pueda registrarse como administrador desde la API pública.

5.3. Autorización por roles

La autorización se basa en la columna `role` de la tabla `users`. Para proteger las rutas de administración se ha creado un middleware específico llamado `EnsureUserIsAdmin`. Este middleware comprueba que el usuario autenticado exista y que su rol sea `admin`.

```
1 class EnsureUserIsAdmin
2 {
3     public function handle(Request $request, Closure $next):
4         Response
5     {
6         if (!$request->user() || $request->user()->role !== '
7             admin') {
8             return response()->json([
9                 'message' => 'No tienes permisos de
10                 administrador.',
11             ], 403);
12         }
13     }
14 }
```

5.4. Rutas públicas y privadas

La API diferencia claramente entre rutas públicas, rutas privadas y rutas privadas de administrador.

- **Públicas:** login, registro, listado de vehículos, detalle de vehículo y disponibilidad.
- **Privadas:** usuario autenticado, logout y reservas.
- **Admin:** creación, edición y borrado de vehículos.

6. Recursos de la API

6.1. Recurso Vehicles

El recurso `vehicles` representa las autocaravanas disponibles en la plataforma. Este recurso se almacena en MySQL y permite operaciones CRUD completas para administradores. Para usuarios públicos, solo se permite consultar vehículos activos.

6.1.1. Campos principales

- `slug`: identificador textual único.
- `name`: nombre del modelo.
- `brand`: marca.
- `price_per_day`: precio por día.
- `description`: descripción.
- `images`: array de URLs de imágenes.
- `features`: array de características.
- `capacity`: capacidad de pasajeros.
- `transmission`: transmisión manual o automática.
- `fuel`: tipo de combustible.
- `is_active`: indica si el vehículo está disponible para el público.

6.1.2. Lógica de negocio

El controlador de vehículos incluye varias reglas de negocio:

- Los usuarios públicos solo ven vehículos activos.
- Los administradores pueden ver también vehículos inactivos.
- El listado permite filtrar por marca, capacidad mínima y precio máximo.
- No se puede eliminar un vehículo con reservas activas.
- Se puede consultar la disponibilidad de un vehículo en un rango de fechas.
- La disponibilidad calcula también los días y el precio estimado.

6.2. Recurso Reservations

El recurso `reservations` representa las reservas de autocaravanas. Este recurso se almacena en MongoDB. Para crear una reserva es obligatorio estar autenticado.

6.2.1. Campos principales

- `user_id`: usuario que crea la reserva.
- `vehicle_id`: vehículo reservado.
- `vehicle_name`: nombre del vehículo en el momento de reservar.
- `start_date`: fecha de inicio.
- `end_date`: fecha de fin.
- `days`: número de días.
- `total_price`: precio total calculado.
- `status`: estado de la reserva.
- `customer_name`: nombre del cliente.
- `customer_email`: email del cliente.
- `customer_phone`: teléfono del cliente.
- `notes`: observaciones opcionales.

6.2.2. Lógica de negocio

El controlador de reservas incorpora lógica específica del problema:

- Solo se puede reservar un vehículo activo.
- No se permiten reservas solapadas para el mismo vehículo.
- El sistema calcula automáticamente los días de reserva.
- El sistema calcula automáticamente el precio total.
- Un usuario normal solo puede ver sus propias reservas.
- Un administrador puede ver todas las reservas.
- Un usuario normal no puede modificar el estado de la reserva.
- Al modificar fechas se recalcula el precio total.

7. Validación de datos

7.1. Uso de Form Request

Para mantener el código organizado, las validaciones se han separado en clases `FormRequest`. Esto evita que los controladores tengan demasiadas reglas mezcladas con la lógica de negocio.

Los principales requests utilizados son:

- `RegisterRequest`
- `LoginRequest`
- `StoreVehicleRequest`
- `UpdateVehicleRequest`
- `StoreReservationRequest`
- `UpdateReservationRequest`

7.2. Validación de vehículos

La creación de vehículos valida campos como nombre, marca, precio, capacidad, transmisión, combustible, imágenes y características.

```
1 public function rules(): array
2 {
3     return [
4         'slug' => 'required|string|max:120|unique:vehicles,slug',
5         'name' => 'required|string|max:120',
6         'brand' => 'required|string|max:80',
7         'price_per_day' => 'required|numeric|min:1|max:9999',
8         'description' => 'required|string|max:2000',
9         'images' => 'nullable|array',
10        'images.*' => 'required_with:images|url',
11        'features' => 'nullable|array',
12        'features.*' => 'required_with:features|string|max:80',
13        ,
14        'capacity' => 'required|integer|min:1|max:10',
15        'transmission' => 'required|string|in:Manual,
16        Automática',
17        'fuel' => 'required|string|max:50',
18        'is_active' => 'sometimes|boolean',
19    ];
20 }
```

7.3. Validación de reservas

La creación de reservas valida que el vehículo exista, que las fechas sean correctas y que los datos del cliente tengan formato válido.

```
1 public function rules(): array
2 {
3     return [
4         'vehicle_id' => 'required|integer|exists:vehicles,id',
5         'start_date' => 'required|date|after_or_equal:today',
6         'end_date' => 'required|date|after_or_equal:start_date',
7         'customer_name' => 'required|string|min:2|max:120',
8         'customer_email' => 'required|email|max:255',
9         'customer_phone' => 'required|string|max:30',
10        'notes' => 'nullable|string|max:1000',
11    ];
12 }
```

8. Endpoints desarrollados

8.1. Autenticación

Método	Endpoint	Acceso	Descripción
POST	/api/register	Público	Registrar usuario
POST	/api/login	Público	Iniciar sesión
GET	/api/me	Privado	Obtener usuario autenticado
POST	/api/logout	Privado	Cerrar sesión

8.2. Vehicles

Método	Endpoint	Acceso	Descripción
GET	/api/vehicles	Público	Listar vehículos
GET	/api/vehicles/{id}	Público	Detalle de vehículo
GET	/api/vehicles/{id}/availability	Público	Consultar disponibilidad
POST	/api/vehicles	Admin	Crear vehículo
PUT/PATCH	/api/vehicles/{id}	Admin	Actualizar vehículo
DELETE	/api/vehicles/{id}	Admin	Eliminar vehículo

8.3. Reservations

Método	Endpoint	Acceso	Descripción
GET	/api/reservations	Privado	Listar reservas
POST	/api/reservations	Privado	Crear reserva
GET	/api/reservations/{id}	Privado	Ver detalle
PATCH	/api/reservations/{id}	Privado	Actualizar reserva
DELETE	/api/reservations/{id}	Privado	Eliminar reserva

9. Códigos HTTP utilizados

La API devuelve códigos HTTP acordes a cada operación. Esto es importante porque una API REST no solo debe devolver datos, sino también indicar correctamente el resultado de la operación.

Código	Uso en la API
200 OK	Peticiones correctas de listado, detalle, login y actualizaciones
201 Created	Creación correcta de usuarios, vehículos o reservas
204 No Content	Eliminación correcta sin cuerpo de respuesta
401 Unauthorized	Credenciales incorrectas o token no enviado
403 Forbidden	Usuario autenticado sin permisos suficientes
404 Not Found	Recurso inexistente o no disponible
409 Conflict	Conflictos de negocio, como reservas solapadas
422 Unprocessable Entity	Errores de validación

Cuadro 5: Códigos HTTP utilizados

10. Instalación y ejecución del proyecto

10.1. Requisitos previos

Para ejecutar el proyecto es necesario tener instalado:

- PHP compatible con Laravel.
- Composer.
- MySQL y phpMyAdmin mediante XAMPP.
- MongoDB Community Server.
- MongoDB Compass.
- Extensión `php_mongodb.dll` habilitada en PHP.
- Thunder Client o Postman para probar la API.

10.2. Configuración de MySQL

En phpMyAdmin se debe crear una base de datos:

```
1 CREATE DATABASE vango_api;
```

Después, en el archivo `.env`:

```
1 DB_CONNECTION=mysql
2 DB_HOST=127.0.0.1
3 DB_PORT=3306
4 DB_DATABASE=vango_api
5 DB_USERNAME=root
6 DB_PASSWORD=
```

10.3. Configuración de MongoDB

MongoDB debe estar ejecutándose en local. La conexión usada por el proyecto es:

```
1 mongodb://127.0.0.1:27017
```

En el archivo `.env` se añaden las variables:

```
1 DB_MONGO_HOST=127.0.0.1
2 DB_MONGO_PORT=27017
3 DB_MONGO_DATABASE=vango_api_mongo
4 DB_MONGO_USERNAME=
5 DB_MONGO_PASSWORD=
6 DB_MONGO_AUTH_DATABASE=admin
```

10.4. Comandos de instalación

Una vez clonado o descargado el proyecto:

```
1 composer install
2 cp .env.example .env
3 php artisan key:generate
4 php artisan optimize:clear
5 php artisan migrate:fresh --seed
6 php artisan serve
```

Si el proyecto ya está configurado localmente, normalmente basta con:

```
1 php artisan optimize:clear
2 php artisan migrate:fresh --seed
3 php artisan serve
```

La API queda disponible en:

```
1 http://127.0.0.1:8000/api
```

11. Tutorial de pruebas con Thunder Client

11.1. Preparación

Para probar la API se ha utilizado Thunder Client. Antes de empezar las pruebas hay que asegurarse de que están levantados los servicios:

- MySQL desde XAMPP.
- MongoDB Server.
- Laravel mediante `php artisan serve`.

La URL base de la API es:

```
1 http://127.0.0.1:8000/api
```

En todas las peticiones se recomienda añadir la cabecera:

```
1 Accept: application/json
```

11.2. Prueba 1: listar vehículos

```
1 GET http://127.0.0.1:8000/api/vehicles
```

Esta ruta es pública y debe devolver el listado de autocaravanas activas.

11.3. Prueba 2: detalle de vehículo

```
1 GET http://127.0.0.1:8000/api/vehicles/1
```

Debe devolver la información completa del vehículo con ID 1.

11.4. Prueba 3: consultar disponibilidad

```
1 GET http://127.0.0.1:8000/api/vehicles/1/availability?
    start_date=2026-07-01&end_date=2026-07-05
```

Respuesta esperada:

```
1 {  
2   "vehicle_id": 1,  
3   "available": true,  
4   "days": 5,  
5   "estimated_price": 925  
6 }
```

Esta prueba permite comprobar la lógica de disponibilidad y el cálculo de precio estimado.

11.5. Prueba 4: login como usuario normal

```
1 POST http://127.0.0.1:8000/api/login
```

Body JSON:

```
1 {  
2   "email": "user@vango.com",  
3   "password": "password"  
4 }
```

La respuesta debe devolver un token. Ese token se usará en las rutas privadas.

11.6. Prueba 5: consultar usuario autenticado

```
1 GET http://127.0.0.1:8000/api/me
```

Cabecera:

```
1 Authorization: Bearer TOKEN_USER
```

Si el token es correcto, la API devuelve los datos del usuario autenticado.

11.7. Prueba 6: crear reserva

```
1 POST http://127.0.0.1:8000/api/reservations
```

Cabeceras:

```
1 Authorization: Bearer TOKEN_USER  
2 Accept: application/json  
3 Content-Type: application/json
```

Body JSON:


```
1 {  
2   "vehicle_id": 1,  
3   "start_date": "2026-07-01",  
4   "end_date": "2026-07-05",  
5   "customer_name": "Carlos Beltran",  
6   "customer_email": "carlos@example.com",  
7   "customer_phone": "600123123",  
8   "notes": "Viaje familiar por el norte"  
9 }
```

La API debe crear una reserva en MongoDB. Además, debe calcular automáticamente los días y el precio total.

11.8. Prueba 7: comprobar reserva solapada

Se vuelve a enviar la misma reserva con las mismas fechas. La API debe responder con un conflicto:

```
1 {  
2   "message": "El veh culo ya est  reservado en ese rango de  
3     fechas."  
4 }
```

El código HTTP esperado es:

```
1 409 Conflict
```

Esta prueba es importante porque demuestra que existe lógica de negocio real.

11.9. Prueba 8: listar reservas del usuario

```
1 GET http://127.0.0.1:8000/api/reservations
```

Cabecera:

```
1 Authorization: Bearer TOKEN_USER
```

Un usuario normal solo debe ver sus propias reservas.

11.10. Prueba 9: actualizar reserva

Para actualizar una reserva se usa el identificador `_id` devuelto por MongoDB.

```
1 PATCH http://127.0.0.1:8000/api/reservations/ID_DE_MONGO
```

Body JSON:

```
1 {  
2   "end_date": "2026-07-06",  
3   "notes": "A ado una noche m s"  
4 }
```

La API debe recalcular los días y el precio total.

11.11. Prueba 10: login como administrador

```
1 POST http://127.0.0.1:8000/api/login
```

Body JSON:

```
1 {  
2   "email": "admin@vango.com",  
3   "password": "password"  
4 }
```

La respuesta devuelve un token de administrador.

11.12. Prueba 11: crear vehículo como administrador

```
1 POST http://127.0.0.1:8000/api/vehicles
```

Cabecera:

```
1 Authorization: Bearer TOKEN_ADMIN
```

Body JSON:

```
1 {  
2   "slug": "hymer-exsis-i-580",  
3   "name": "Hymer Exsis-i 580",  
4   "brand": "Hymer",  
5   "price_per_day": 195,  
6   "description": "Autocaravana integral ligera con gran  
7     aislamiento y distribuci n c moda para rutas largas.",  
8   "images": [  
9     "https://example.com/hymer-1.jpg",  
10    "https://example.com/hymer-2.jpg"  
11  ],  
12  "features": [  
13    "Cama isla",  
14    "Garaje amplio",  
    "Calefacci n",  
  ]  
}
```

```
15     "Placa solar"
16   ],
17   "capacity": 4,
18   "transmission": "Autom tica",
19   "fuel": "Di sel",
20   "is_active": true
21 }
```

La respuesta esperada es 201 Created.

11.13. Prueba 12: comprobar autorización

Se repite la creación de vehículo usando el token de usuario normal. La API debe devolver:

```
1 {
2   "message": "No tienes permisos de administrador."
3 }
```

El código esperado es 403 Forbidden.

11.14. Prueba 13: eliminar vehículo

```
1 DELETE http://127.0.0.1:8000/api/vehicles/5
```

Cabecera:

```
1 Authorization: Bearer TOKEN_ADMIN
```

Si el vehículo no tiene reservas activas, la respuesta debe ser:

```
1 204 No Content
```

Si tiene reservas activas, la API debe responder con 409 Conflict.

12. Pruebas realizadas

12.1. Pruebas de rutas públicas

Se han probado las rutas públicas de vehículos para comprobar que se puede acceder al catálogo sin autenticación. Estas rutas permiten listar vehículos, ver el detalle de un vehículo concreto y consultar disponibilidad. También se ha comprobado que los vehículos inactivos no se muestran a usuarios públicos.

12.2. Pruebas de autenticación

Se han probado el registro, el login, la consulta del usuario autenticado y el logout. La autenticación se considera correcta cuando el usuario recibe un token y puede acceder con él a rutas protegidas.

12.3. Pruebas de autorización

Se ha probado que los usuarios normales no puedan crear, actualizar o eliminar vehículos. También se ha comprobado que el administrador sí puede realizar esas acciones. Esto demuestra que el middleware de administración funciona correctamente.

12.4. Pruebas de reservas

Se han realizado pruebas creando reservas válidas, intentando crear reservas solapadas, listando reservas del usuario y modificando fechas. También se ha comprobado que el sistema recalcula correctamente el número de días y el precio total al actualizar una reserva.

12.5. Comprobación en bases de datos

Para MySQL se ha utilizado phpMyAdmin, verificando las tablas `users`, `vehicles` y `personal_access_tokens`. Para MongoDB se ha utilizado MongoDB Compass, comprobando que las reservas se almacenan en la base de datos `vango_api_mongo`, dentro de la colección `reservations`.

13. Problemas encontrados y soluciones

13.1. Integración de MongoDB

Uno de los puntos más delicados fue la instalación de MongoDB y su integración con PHP. No bastaba con instalar MongoDB Compass, ya que Compass solo es una interfaz visual. También era necesario instalar MongoDB Community Server y activar la extensión `php_mongodb.dll` en XAMPP.

La solución fue instalar MongoDB Server, comprobar la conexión con Compass, descargar la DLL correcta desde PECL, copiarla en `C:\xampp\php\ext` y añadir la extensión en el archivo `php.ini`.

13.2. Diferencia entre MySQL y MongoDB

Otro punto importante fue entender que no hay claves foráneas reales entre MySQL y MongoDB. Por eso, las reservas guardan `user_id` y `vehicle_id` como referencias lógicas. La integridad de esas relaciones se controla desde la lógica de la aplicación.

13.3. Control de permisos

También fue necesario diferenciar autenticación y autorización. Un usuario autenticado no tiene por qué ser administrador. Para resolver esto se creó un middleware específico de admin que protege las rutas de gestión de vehículos.

13.4. Solapamiento de reservas

Una parte importante de la lógica fue evitar que dos usuarios pudieran reservar el mismo vehículo en fechas solapadas. Esta comprobación se realiza antes de crear o actualizar una reserva, consultando en MongoDB si ya existe una reserva pendiente o confirmada para el mismo vehículo y rango de fechas.

14. Conclusiones y mejoras futuras

14.1. Conclusiones

El desarrollo de esta API ha permitido convertir la idea inicial de VanGo en una aplicación backend más cercana a un producto real. La parte frontend simulaba los datos mediante un archivo JSON, mientras que este backend introduce persistencia real, autenticación, roles, validación, lógica de negocio y separación entre rutas públicas y privadas.

El resultado es una API REST organizada, con dos recursos principales completos y una arquitectura coherente con la temática del alquiler de autocaravanas. Además, el uso combinado de MySQL y MongoDB permite cumplir los requisitos técnicos del módulo y al mismo tiempo justificar una separación lógica entre datos estructurados y datos documentales.

14.2. Mejoras futuras

Como mejoras futuras, sería interesante conectar el frontend desarrollado en el módulo anterior con esta API, sustituyendo el archivo JSON local por llamadas reales al backend. También se podría añadir un sistema de estados más completo para las reservas, integración con pagos, calendario visual de disponibilidad y subida real de imágenes de vehículos.

Otra mejora posible sería añadir documentación Swagger/OpenAPI para que la API se pudiera explorar de forma más cómoda. También se podrían incluir tests automáticos para validar los casos principales de negocio, como la creación de reservas, la autorización de administradores y la comprobación de disponibilidad.